

CS 224n Assignment #2: Word2Vec and Dependency Parsing

截止日期: 4 月 18 日 (周四), 美国太平洋时间下午 4:30。

在这个作业中, 你将复习 Word2Vec 背后的数学原理, 并使用 PyTorch 构建一个神经依存句法分析器。关于 PyTorch 基础知识的复习, 请查看 Canvas 上的 PyTorch 复习课。第 1 部分中, 你将探索使用 naive softmax loss 训练 Word2Vec 模型时涉及的偏导数。第 2 部分中, 你将学习两种通用神经网络技巧 (Adam 优化和 Dropout)。第 3 部分中, 你将实现并训练一个依存句法分析器, 并使用第 2 部分中的技术, 在分析几个错误依存句法解析的同时完成训练。

如果你使用 LaTeX, 可以使用 `\ifans{}` 来输入你的解答。

请在 Gradescope 上正确标记题目, 否则如果你没有正确标记, 助教会扣分。

1. 理解 word2vec (15 points)

回想一下 word2vec 背后的关键洞见是: “一个词是由它周围的词所定义的”。具体来说, 考虑一个“中心词” c , 它前后都有一定长度的上下文。我们将这个上下文窗口中的词称为“外部词” (O)。例如, 在图 1 中, 上下文窗口长度为 2, 中心词 c 是 ‘banking’, 外部词分别是 ‘turning’、‘into’、‘crises’ 和 ‘as’:

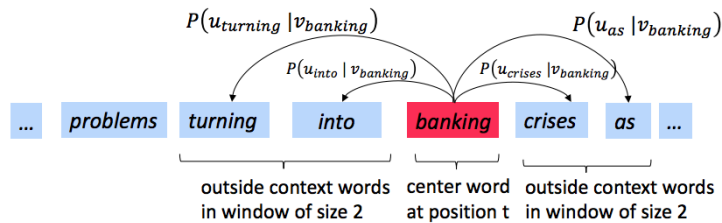


图 1: 窗口大小为 2 的 word2vec skip-gram 预测模型

Skip-gram word2vec 旨在学习概率分布 $P(O|C)$ 。具体地说, 给定一个特定词 o 和一个特定词 c , 我们想预测 $P(O = o | C = c)$: 词 o 是 c 的“外部词”的概率 (也就是说, 它落在 c 的上下文窗口中)。我们通过对一系列向量点积取 softmax 来建模这个概率:

$$P(O = o | C = c) = \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{w \in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c)} \quad (1)$$

对每个词, 我们学习两个向量 u 和 v , 其中 $\mathbf{u}_o$ 是表示外部词 o 的“外部”向量, $\mathbf{v}_c$ 是表示中心词 c 的“中心”向量。我们把这些参数存储在两个矩阵 $\mathbf{U}$ 和 $\mathbf{V}$ 中。 $\mathbf{U}$ 的列是所有“外部”向量 $\mathbf{u}_w$; $\mathbf{V}$ 的列是所有“中心”向量 $\mathbf{v}_w$ 。 $\mathbf{U}$ 和 $\mathbf{V}$ 都为词表中的每个 $w \in \text{Vocabulary}$ 保存一个向量。¹

回顾课堂内容, 对于一对单词 c 和 o , 损失函数为:

$$\mathbf{J}_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U}) = -\log P(O = o | C = c). \quad (2)$$

我们可以把这个损失看成真实分布 $\mathbf{y}$ 与预测分布 $\hat{\mathbf{y}}$ 之间的交叉熵², 对应于特定中心词 c 和特定外部词 o 。这里, $\mathbf{y}$ 和 $\hat{\mathbf{y}}$ 都是长度等于词表大小的向量。进一步地, 这些向量的第 k 项表示给定中心词 c 时, 第 k 个词成为“外部词”的条件概率。真实经验分布 $\mathbf{y}$ 是一个 one-hot 向量: 对于真实外

¹假设词表中的每个词都对应一个整数编号 k 。粗体小写字母表示向量。 $\mathbf{u}_k$ 同时表示 $\mathbf{U}$ 的第 k 列, 以及索引为 k 的词对应的“外部词”向量。 $\mathbf{v}_k$ 同时表示 $\mathbf{V}$ 的第 k 列, 以及索引为 k 的词对应的“中心词”向量。为了简化记号, 我们将交替使用 k 表示词 k 以及词 k 的索引。

²真实 (离散) 概率分布 p 与另一个分布 q 之间的交叉熵损失是 $-\sum_i p_i \log(q_i)$ 。

部词 o ，对应位置为 1，其余位置为 0，这对应于某个特定中心词 c 和外部词 o 的一个样本。³ 预测分布 $\hat{\mathbf{y}}$ 是我们模型在式 (1) 中给出的概率分布 $P(O|C = c)$ 。

注意：在本作业中，计算导数时，请使用课堂中复习过的方法（即不要使用泰勒级数近似）。

³注意，整个训练数据的真实条件概率分布并不一定是 one-hot。

- (a) (2 points) 证明 naive-softmax loss (式 2) 与 $\mathbf{y}$ 和 $\hat{\mathbf{y}}$ 之间的交叉熵损失相同, 也就是 (注意 $\mathbf{y}$ (真实分布)、 $\hat{\mathbf{y}}$ (预测分布) 都是向量, $\hat{\mathbf{y}}_o$ 是标量):

$$\mathbf{y}_w = \begin{cases} 1, & w = o, \\ 0, & w \neq o, \end{cases} \Rightarrow - \sum_{w \in \text{Vocab}} \mathbf{y}_w \log(\hat{\mathbf{y}}_w) = -\log(\hat{\mathbf{y}}_o). \quad (3)$$

因为真实分布 $\mathbf{y}$ 是 one-hot 向量, 只有真实外部词 o 那一项为 1, 其余为 0, 所以交叉熵只保留第 o 项; 而 $\hat{\mathbf{y}}_o = P(O = o \mid C = c)$, 因此

$$J_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U}) = -\log P(O = o \mid C = c) = -\log(\hat{\mathbf{y}}_o) = - \sum_{w \in \text{Vocab}} \mathbf{y}_w \log(\hat{\mathbf{y}}_w).$$

- (b) (6 points) i. 计算 $\mathbf{J}_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U})$ 相对于 $\mathbf{v}_c$ 的偏导数。请用 $\mathbf{y}$ 、 $\hat{\mathbf{y}}$ 、 $\mathbf{U}$ 的形式写出答案, 并展示你的推导过程以获得满分。

- **注意:** 最终偏导数的答案应遵循形状约定: 任意函数 $f(x)$ 相对于 x 的偏导数必须与 x 具有**相同形状**。⁴
- 请以向量化形式给出偏导数答案。例如, 当我们要求你用 $\mathbf{y}$ 、 $\hat{\mathbf{y}}$ 和 $\mathbf{U}$ 表达答案时, 你在最终答案中不能引用这些项的特定元素 (例如 $\mathbf{y}_1$ 、 $\mathbf{y}_2$ 、 $\dots$)。

- ii. 当你计算出的梯度等于零时, 什么情况下会发生?

提示: 你可能需要复习并使用一些基础线性代数概念。

- iii. 你找到的梯度是两个项之差。请解释这两个项分别如何在将梯度从词向量 $\mathbf{v}_c$ 中减去时改善词向量。

- (c) (1 point) 在许多使用词嵌入的下游应用中, 通常使用 L2 归一化向量 (例如 $\mathbf{u}/\|\mathbf{u}\|_2$, 其中 $\|\mathbf{u}\|_2 = \sqrt{\sum_i u_i^2}$) 而不是原始形式 (例如 $\mathbf{u}$)。考虑一个假设的二分类下游任务: 判断短语是正向还是负向, 你根据单词嵌入的和来决定符号。在什么情况下, L2 归一化会丢失有用信息? 在什么情况下不会?

提示: 考虑 $\mathbf{u}_x = \alpha \mathbf{u}_y$ 的情况, 其中 $x \neq y$ 且 α 是某个标量。当 α 为正时, 归一化后的 $\mathbf{u}_x$ 和 $\mathbf{u}_y$ 会是什么? 如果这类归一化会影响或不会影响最终分类, $\mathbf{u}_x$ 和 $\mathbf{u}_y$ 可能具有什么关系?

- (d) (5 points) 计算 $\mathbf{J}_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U})$ 相对于每个“外部词”向量 $\mathbf{u}_w$ 的偏导数。分两种情况: 当 $w = o$ 时, 这是真实的“外部词”向量; 当 $w \neq o$ 时, 这是其他所有词的向量。请用 $\mathbf{y}$ 、 $\hat{\mathbf{y}}$ 和 $\mathbf{v}_c$ 表达答案。在这一小问中, 你也可以引用这些项中的具体元素 (例如 $\mathbf{y}_1$ 、 $\mathbf{y}_2$ 、 $\dots$)。注意, $\mathbf{u}_w$ 是一个向量, 而 $\mathbf{y}_1, \mathbf{y}_2, \dots$ 是标量。展示你的推导过程以获得满分。

- (e) (1 point) 写出 $\mathbf{J}_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U})$ 相对于 $\mathbf{U}$ 的偏导数。请按列向量 $\frac{\partial \mathbf{J}(\mathbf{v}_c, o, \mathbf{U})}{\partial \mathbf{u}_1}$ 、 $\frac{\partial \mathbf{J}(\mathbf{v}_c, o, \mathbf{U})}{\partial \mathbf{u}_2}$ 、 $\dots$ 、 $\frac{\partial \mathbf{J}(\mathbf{v}_c, o, \mathbf{U})}{\partial \mathbf{u}_{|\text{Vocab}|}}$ 来分解答案。不需要推导, 只需给出符合矩阵形式的答案。

⁴这样可以让使用梯度下降高效最小化函数, 而不必担心重塑或维度不匹配。在遵守形状约定的情况下, 我们保证 $\theta := \theta - \alpha \frac{\partial J(\theta)}{\partial \theta}$ 是一个良定义的更新规则。

2. 机器学习 & 神经网络 (8 points)

(a) (4 points) Adam 优化器

回想标准随机梯度下降 (SGD) 的更新规则:

$$\theta_{t+1} \leftarrow \theta_t - \alpha \nabla_{\theta_t} J_{\text{minibatch}}(\theta_t)$$

其中 $t+1$ 是当前时间步, θ 是包含所有模型参数的向量 (θ_t 表示时间步 t 时的模型参数, θ_{t+1} 表示时间步 $t+1$ 时的模型参数), J 是损失函数, $\nabla_{\theta_t} J_{\text{minibatch}}(\theta_t)$ 是损失函数对 minibatch 数据上参数的梯度, α 是学习率。Adam 优化⁵ 使用更复杂的更新规则, 额外包含两步。⁶

i. (2 points) 首先, Adam 使用了一种叫做 *momentum* 的技巧, 通过维护 $\mathbf{m}$ 来记录梯度的滚动平均:

$$\mathbf{m}_{t+1} \leftarrow \beta_1 \mathbf{m}_t + (1 - \beta_1) \nabla_{\theta_t} J_{\text{minibatch}}(\theta_t)$$

$$\theta_{t+1} \leftarrow \theta_t - \alpha \mathbf{m}_{t+1}$$

其中 β_1 是介于 0 和 1 之间的超参数 (通常设为 0.9)。请用 2-4 句话简要解释 (不需要严格证明, 只需给出直觉): $\mathbf{m}$ 是如何让更新的波动变小的, 以及这种低方差为什么可能有助于整体学习。

ii. (2 points) Adam 在 *momentum* 的基础上进一步引入了 *adaptive learning rates* 的思想, 即维护 $\mathbf{v}$, 它是梯度大小的滚动平均:

$$\mathbf{m}_{t+1} \leftarrow \beta_1 \mathbf{m}_t + (1 - \beta_1) \nabla_{\theta_t} J_{\text{minibatch}}(\theta_t)$$

$$\mathbf{v}_{t+1} \leftarrow \beta_2 \mathbf{v}_t + (1 - \beta_2) (\nabla_{\theta_t} J_{\text{minibatch}}(\theta_t) \odot \nabla_{\theta_t} J_{\text{minibatch}}(\theta_t))$$

$$\theta_{t+1} \leftarrow \theta_t - \alpha \mathbf{m}_{t+1} / \sqrt{\mathbf{v}_{t+1}}$$

其中 $\odot$ 和 $/$ 分别表示逐元素乘法和除法 (因此 $\mathbf{z} \odot \mathbf{z}$ 表示逐元素平方), β_2 是介于 0 和 1 之间的超参数 (通常设为 0.99)。由于 Adam 会用 $\sqrt{\mathbf{v}}$ 去除更新, 哪些模型参数会得到更大的更新? 为什么这可能有助于学习?

(b) (4 points) Dropout⁷ 是一种正则化技术。在训练期间, dropout 会以概率 p_{drop} 随机将隐藏层 $\mathbf{h}$ 中的单元置为 0 (每个 minibatch 可能丢弃不同的单元), 然后再乘上一个常数 γ 。我们可以写成:

$$\mathbf{h}_{\text{drop}} = \gamma \mathbf{d} \odot \mathbf{h}$$

其中 $\mathbf{d} \in \{0, 1\}^{D_h}$ (D_h 是 $\mathbf{h}$ 的大小) 是一个掩码向量, 其中每个元素以概率 p_{drop} 取 0, 并以概率 $(1 - p_{\text{drop}})$ 取 1。 γ 的选择要满足 $\mathbf{h}_{\text{drop}}$ 的期望值等于 $\mathbf{h}$:

$$\mathbb{E}_{p_{\text{drop}}}[\mathbf{h}_{\text{drop}}]_i = h_i$$

对所有 $i \in \{1, \dots, D_h\}$ 成立。

- (2 points) γ 必须等于多少 (用 p_{drop} 表示)? 简要说明理由或利用给定公式给出数学推导。
- (2 points) 为什么训练时应使用 dropout? 为什么在评估时不应使用 dropout? 提示: 参考链接的 dropout 论文可能会有帮助。

⁵Kingma and Ba, 2015, <https://arxiv.org/pdf/1412.6980.pdf>

⁶真实的 Adam 更新还包含一些次要技巧, 但这里我们不考虑它们。如果你想了解更多, 可以查看: <http://cs231n.github.io/neural-networks-3/#sgd>

⁷Srivastava et al., 2014, <https://www.cs.toronto.edu/~hinton/absps/JMLRdropout.pdf>

3. 基于转移的神经依存句法分析 (54 points)

在这一部分，你将实现一个基于神经网络的依存句法分析器，目标是最大化 UAS (Unlabeled Attachment Score, 无标签依存附着分数) 指标。

在开始之前，请按照 README 中的说明安装该作业所需的所有依赖。我们将使用来自 <https://pytorch.org/get-started/locally/> 的 PyTorch 2.1.2，并将 CUDA 选项设为 None；同时使用 tqdm 包，它会在训练过程中显示进度条。PyTorch 官方网站提供了大量教程，对理解 PyTorch 的 Tensor 库和神经网络非常有帮助。

依存句法分析器会分析句子的语法结构，建立 *head* (核心词) 和修饰这些核心词的词之间的关系。依存句法分析器有多种类型，包括转移式分析器、基于图的分析器和基于特征的分析器。你的实现将是一个转移式分析器，它会逐步增量构建语法分析结果。每一步都维护一个 *partial parse* (部分分析)，其表示如下：

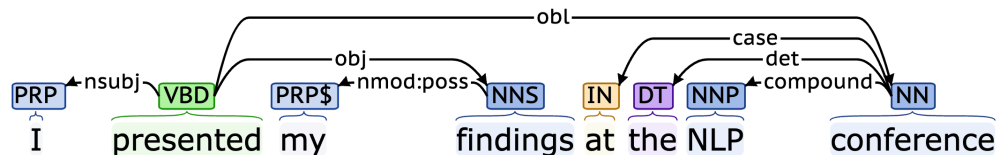
- 一个当前正在处理的词 *stack* (栈)。
- 一个尚未处理的词 *buffer* (缓冲区)。
- 一个由分析器预测出的 *dependencies* (依存关系) 列表。

初始时，栈中只包含 ROOT，依存关系列表为空，缓冲区按原句顺序包含所有词。每一步，分析器都会对当前部分分析应用一个 *transition* (转移)，直到缓冲区为空且栈大小为 1。可以应用的转移如下：

- **SHIFT**: 从缓冲区中移除第一个词并压入栈顶。
- **LEFT-ARC**: 将栈中第二个 (最近加入的第二个) 元素标记为第一个元素的从属词，并从栈中移除这个第二个元素，将一条 *first_word* $\rightarrow$ *second_word* 的依存关系加入依存关系列表。
- **RIGHT-ARC**: 将栈中第一个 (最近加入的) 元素标记为第二个元素的从属词，并从栈中移除第一个元素，将一条 *second_word* $\rightarrow$ *first_word* 的依存关系加入依存关系列表。

每一步，分析器都会使用神经网络分类器在这三种转移中做出选择。

- (a) (4 points) 按照解析句子 “I presented my findings at the NLP conference” 所需的转移顺序逐步操作。该句子的依存树如下图所示。每一步都给出栈和缓冲区的配置，以及该步应用了什么转移，以及新增了什么依存关系 (如果有)。前三步已作为示例给出。



Stack	Buffer	New dependency	Transition
[ROOT]	[I, presented, my, findings, at, the, NLP, conference]		Initial Configuration
[ROOT, I]	[presented, my, findings, at, the, NLP, conference]		SHIFT
[ROOT, I, presented]	[my, findings, at, the, NLP, conference]		SHIFT
[ROOT, presented]	[my, findings, at, the, NLP, conference]	presented $\rightarrow$ I	LEFT-ARC

- (b) (2 points) 一个包含 n 个词的句子需要多少步来完成解析 (用 n 表示)? 请用 1-2 句话说明原因。

- (c) (6 points) 在 `PartialParse` 类中实现 `__init__` 和 `parse_step` 函数, 位置在 `parser_transitions.py`。它实现了分析器使用的转移机制。你可以运行 `python parser_transitions.py part_c` 来进行基础（非穷尽）测试。
- (d) (8 points) 我们的网络将预测每个部分分析下一步应该应用哪个转移。我们可以用它逐个句子解析：持续应用模型预测的转移，直到解析完成；不过，神经网络在对批量数据做预测时效率更高（即同时预测多个不同部分分析的下一步转移）。我们可以使用下面的算法对句子进行 minibatch 解析。

Algorithm 1 Minibatch Dependency Parsing

Input: `sentences`, 待解析句子的列表, 以及 `model`, 用于做解析决策的模型

将 `partial_pares` 初始化为一组 `PartialPares`, 每个句子对应一个

将 `unfinished_pares` 初始化为 `partial_pares` 的浅拷贝

while `unfinished_pares` 非空 **do**

 取 `unfinished_pares` 中前 `batch_size` 个解析作为一个 minibatch

 用 `model` 对 minibatch 中每个部分分析预测下一步转移

 对 minibatch 中每个部分分析使用它预测到的转移执行一次解析步骤

 从 `unfinished_pares` 中删除已完成的解析（缓冲区为空且栈大小为 1）

end while

Return: `partial_pares` 中每个（现已完成）解析的 `dependencies`。

在 `parser_transitions.py` 中实现 `minibatch_parse` 函数。你可以运行 `python parser_transitions.py part_d` 进行基础（非穷尽）测试。

注意：你需要正确实现 `minibatch_parse`，才能评估你在第 (e) 部分中构建的模型；不过，你不需要先训练模型就能完成第 (e) 部分的大部分内容，所以即使 `minibatch_parse` 还没实现好，你也可以先完成大部分工作。

- (e) (20 points) 现在我们要训练一个神经网络，用于根据栈、缓冲区和依存关系状态，预测下一步应该应用哪个转移。

首先，模型会提取一个表示当前状态的特征向量。我们将使用原始神经依存句法分析论文中提出的特征集： *A Fast and Accurate Dependency Parser using Neural Networks*.⁸ 提取这些特征的函数已经为你实现好，位于 `utils/parser_utils.py`。这个特征向量由一组 token 组成（例如栈顶的最后一个词、缓冲区开头的第一个词、栈倒数第二个词的从属词等）。它们可以表示为一个整数列表 $\mathbf{w} = [w_1, w_2, \dots, w_m]$ ，其中 m 是特征的数量，且每个 $0 \leq w_i < |V|$ 表示词表中某个 token 的索引（ $|V|$ 是词表大小）。然后我们的网络会为每个词查表得到一个 embedding，并将这些 embedding 拼接成一个输入向量：

$$\mathbf{x} = [\mathbf{E}_{w_1}, \dots, \mathbf{E}_{w_m}] \in \mathbb{R}^{dm}$$

其中 $\mathbf{E} \in \mathbb{R}^{|V| \times d}$ 是 embedding 矩阵，每一行 $\mathbf{E}_w$ 是某个词 w 的向量，维度为 d 。然后我们计

⁸Chen and Manning, 2014, <https://nlp.stanford.edu/pubs/emnlp2014-depparser.pdf>

算预测：

$$\mathbf{h} = \text{ReLU}(\mathbf{x}\mathbf{W} + \mathbf{b}_1)$$

$$\mathbf{l} = \mathbf{h}\mathbf{U} + \mathbf{b}_2$$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{l})$$

其中 $\mathbf{h}$ 被称为隐藏层, $\mathbf{l}$ 被称为 logits, $\hat{\mathbf{y}}$ 被称为预测值, 且 $\text{ReLU}(z) = \max(z, 0)$ 。我们将训练模型来最小化交叉熵损失：

$$J(\theta) = CE(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{j=1}^3 \mathbf{y}_j \log \hat{\mathbf{y}}_j$$

其中 $\mathbf{y}_j$ 表示 $\mathbf{y}$ 的第 j 项。为了计算训练集的损失, 我们对所有训练样本的这个 $J(\theta)$ 取平均。

- 计算 $\mathbf{h} = \text{ReLU}(\mathbf{x}\mathbf{W} + \mathbf{b}_1)$ 相对于 $\mathbf{x}$ 的导数。为简化起见, 只需要展示某个索引 i 和 j 的导数 $\frac{\partial h_i}{\partial x_j}$ 即可; 你可以忽略在 0 处不可导的情形。
- 回想第 1b 部分, 我们计算过 $\mathbf{J}_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U})$ 的偏导数。类似地, 请计算 $J(\theta)$ 相对于 $\mathbf{l}$ 的第 i 项 $\mathbf{l}_i$ 的偏导数, 即计算 $\frac{\partial CE(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{l}_i}$, 假设 $\mathbf{l} \in \mathbb{R}^3$, $\hat{\mathbf{y}} \in \mathbb{R}^3$, $\mathbf{y} \in \mathbb{R}^3$, 真实标签为 c 。
提示: 你可能还记得第 1a 部分中, $\frac{\partial CE(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{l}_i} = \sum_j \frac{\partial CE(\mathbf{y}, \hat{\mathbf{y}})}{\partial \hat{\mathbf{y}}_j} \frac{\partial \hat{\mathbf{y}}_j}{\partial \mathbf{l}_i}$, 并且如果 $j \neq c$, 则 $\frac{\partial CE(\mathbf{y}, \hat{\mathbf{y}})}{\partial \hat{\mathbf{y}}_j} = 0$ 。
- 我们将使用 UAS 作为评估指标。UAS 指的是 Unlabeled Attachment Score, 它等于正确预测的依存关系数量除以总依存关系数量, 而不考虑关系类型 (因为我们的模型不预测这些标签)。

在 `parser_model.py` 中, 你会找到用 PyTorch 实现这个简单神经网络的骨架代码。完成 `__init__`、`embedding_lookup` 和 `forward` 函数以实现该模型。随后在 `run.py` 中完成 `train_for_epoch` 和 `train` 函数。

最后执行 `python run.py` 以训练模型, 并在 Penn Treebank (用 Universal Dependencies 标注) 的测试数据上计算预测结果。

注意:

- 本作业要求你实现 Linear layer 和 Embedding layer。请**不要**在代码中使用 `torch.nn.Linear` 或 `torch.nn.Embedding` 模块, 否则会在本题中扣分。
- 如果有命名要求, 请遵循 TODO 中的要求。例如, 如果明确要求变量名, 你必须按要求命名以获得满分; 若没有显式要求, 你可以声明其他变量名。

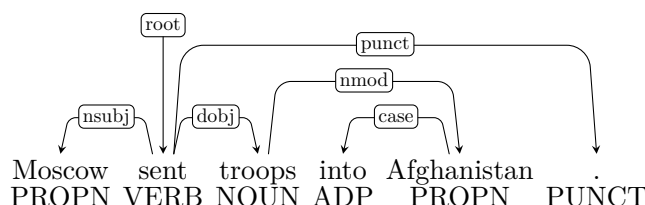
提示:

- 你需要声明的每个变量 (`self.embed_to_hidden_weight`、`self.embed_to_hidden_bias`、`self.hidden_to_logits_weight`、`self.hidden_to_logits_bias`) 对应于上文中的一个变量 ($\mathbf{W}$ 、 $\mathbf{b}_1$ 、 $\mathbf{U}$ 、 $\mathbf{b}_2$)。
- 最好从算法的后面往前推 (从 $\hat{\mathbf{y}}$ 开始), 并持续跟踪矩阵和向量的尺寸。
- 一旦你实现了 `embedding_lookup` (`e`) 或 `forward` (`f`), 就可以使用 `python parser_model.py` 加上 `-e` 或 `-f` (或者两者一起) 来对各个函数运行 sanity check。这些 sanity check 很基础, 能通过并不代表代码完全没有 bug。
- 调试时, 你可以添加 debug 标志: `python run.py -d`。这样代码只会跑一小部分数据, 因此训练模型不会花太长时间。调试完成后, 请移除 `-d` 标志, 以运行完整模型。
- 在 debug 模式下, 你应该能够在验证集上获得小于 0.2 的 loss 和大于 65 的 UAS (尽管有时可能更低, 因为训练存在一定随机性)。

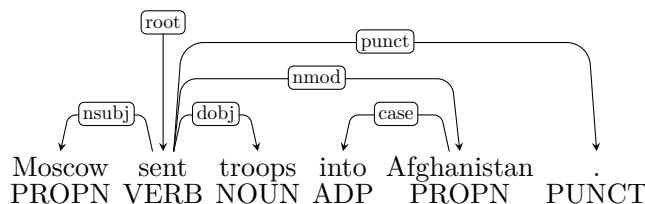
- 在不使用 debug 模式时，整个训练数据集最多可能需要约 **15 分钟**。
- 在不使用 debug 模式时，你应该能够在训练集上获得小于 0.08 的 loss，并在开发集上获得大于 87 的 Unlabeled Attachment Score。作为比较，原始神经依存句法分析论文中的模型可达到 92.5 UAS。如果你愿意，可以微调模型超参数（隐藏层大小、Adam 超参数、epoch 数量等）来提高性能（但这不是必需的）。

交付物：

- 在 `parser_transitions.py` 中实现神经依存句法分析器所用转移机制的可运行代码。
 - 在 `parser_transitions.py` 中实现 minibatch 依存句法分析的可运行代码。
 - 在 `parser_model.py` 中实现神经依存句法分析器的可运行代码。（我们会查看并运行此代码进行评分）。
 - 在 `run.py` 中实现训练函数的可运行代码。（我们会查看并运行此代码进行评分）。
 - 在你的书面提交中报告模型在开发集上达到的最佳 UAS，以及它在测试集上的 UAS。
- (f) (12 points) 我们想看看示例依存句法分析，并理解像我们这样的分析器可能在哪里出错。例如，在下面这个句子中：



短语 *into Afghanistan* 的依存关系是错误的，因为这个短语应该修饰 *sent*（如 *sent into Afghanistan*），而不是 *troops*（因为 *troops into Afghanistan* 这个说法没有意义，除非某些士兵莫名其妙地被描述成“喜欢 Afghanistan”）。下面是正确的分析：



更一般地，下面是四类常见的解析错误：

- **Prepositional Phrase Attachment Error** (介词短语附着错误)：在上面的例子中，短语 *into Afghanistan* 是一个介词短语⁹。介词短语附着错误指的是一个介词短语被附着到了错误的核心词上（在这个例子中，*troops* 是错误的核心词，而 *sent* 才是正确的核心词）。介词短语的其他例子包括 *with a rock*、*before midnight* 和 *under the carpet*。
- **Verb Phrase Attachment Error** (动词短语附着错误)：在句子 *Leaving the store unattended, I went outside to watch the parade* 中，短语 *leaving the store unattended* 是一个动词短语¹⁰。动词短语附着错误指的是一个动词短语被附着到了错误的核心词上（在这个例子中，正确的核心词是 *went*）。
- **Modifier Attachment Error** (修饰语附着错误)：在句子 *I am extremely short* 中，副词 *extremely* 修饰形容词 *short*。修饰语附着错误指的是一个修饰语被附着到了错误的核心词上（在这个例子中，正确的核心词是 *short*）。

⁹ 关于介词短语的例子，可参考：<https://www.grammarly.com/blog/prepositional-phrase/>

¹⁰ 关于动词短语的例子，可参考：<https://examples.yourdictionary.com/verb-phrase-examples.html>

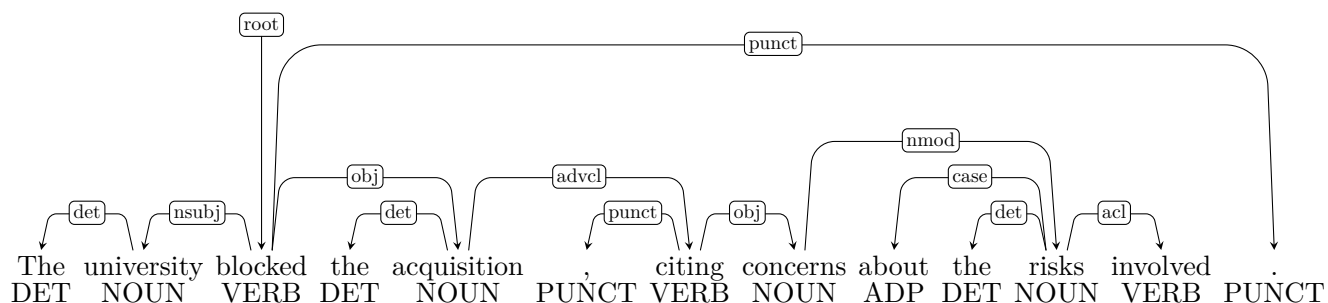
- **Coordination Attachment Error** (并列附着错误): 在句子 *Would you like brown rice or garlic naan?* 中, 短语 *brown rice* 和 *garlic naan* 都是并列成分, 单词 *or* 是并列连词。第二个并列成分 (这里是 *garlic naan*) 应该附着到第一个并列成分 (这里是 *brown rice*) 上。并列附着错误指的是第二个并列成分被附着到了错误的核心词上 (在这个例子中, 正确的核心词是 *rice*)。其他并列连词包括 *and*、*but* 和 *so*。

这一题中有四个句子, 每个句子都有一个依存句法分析结果, 并且它们各自对应上面四类错误中的一种。对于每个句子, 说明错误类型、不正确的依存关系和正确的依存关系。虽然每个句子应具有唯一的错误类型, 但某些句子的正确依存关系可能不唯一。为了说明, 针对上面的例子, 你可以写:

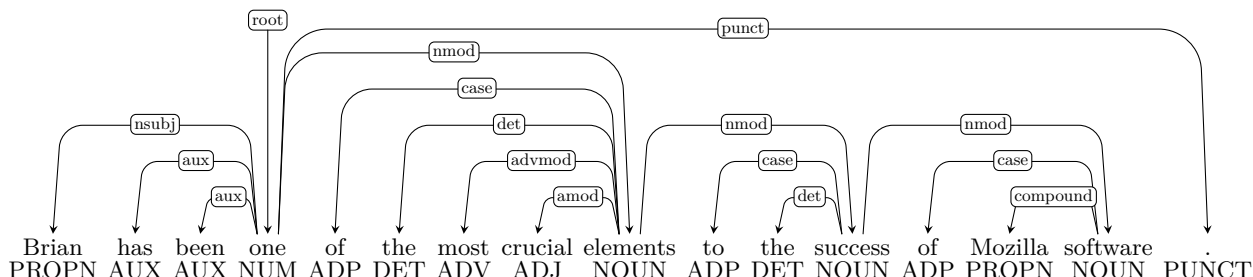
- **Error type:** Prepositional Phrase Attachment Error
- **Incorrect dependency:** troops → Afghanistan
- **Correct dependency:** sent → Afghanistan

注意: 依存关系标注有很多细节和约定。如果你想了解更多, 可以查看 UD 网站: [*http://universaldependencies.org*](http://universaldependencies.org)¹¹, 或者查看简短的入门幻灯片: [*http://people.cs.georgetown.edu/nlschneid/p/UD-for-English.pdf*](http://people.cs.georgetown.edu/nlschneid/p/UD-for-English.pdf)。注意, 你不需要知道所有这些细节来完成这一题。在这些例子中, 我们需要关注的是短语的附着关系, 只要看它们是否修饰了正确的核心词即可。特别地, 你不需要看依存边上的标签——只需看边本身即可。

i.

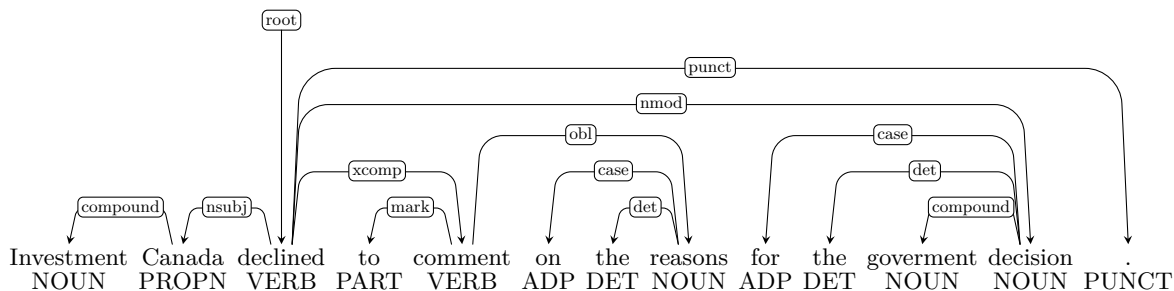


ii.

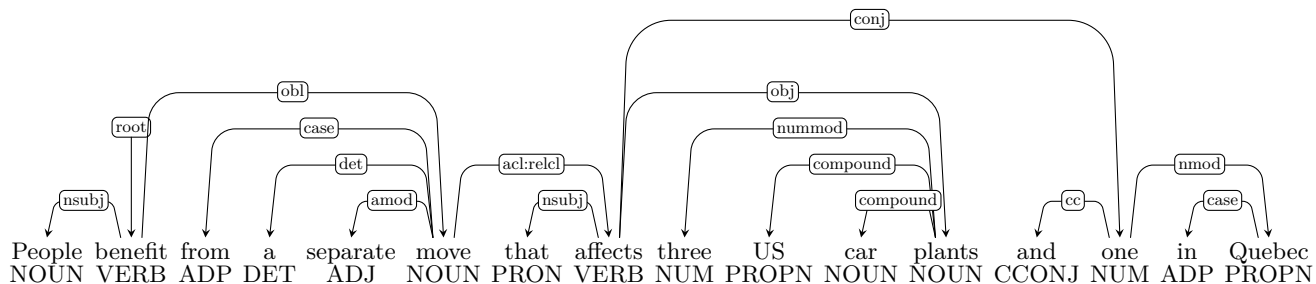


iii.

¹¹但请注意, 本作业实际上使用的是 UDv1, 见: <http://universaldependencies.org/docsv1/>



iv.



- (g) (2 points) 回想第 (e) 部分，分析器使用的特征包括词和它们的词性 (POS) 标签。解释在分析器中使用词性标签作为特征有什么好处？

提交说明

你需要在 GradeScope 上提交两次作业——一份为 “Assignment 2 [coding]”，另一份为 “Assignment 2 [written]”：

1. 运行 `collect_submission.sh` 脚本，生成你的 `assignment2.zip` 文件。
2. 将 `assignment2.zip` 文件上传到 GradeScope 的 “Assignment 2 [coding]”。
3. 将你的书面解答上传到 GradeScope 的 “Assignment 2 [written]”。